LifeSync

Program Documentation

A Java desktop application for wellness, nutrition, fitness, and daily planning.

Plan. Eat. Move. Grow.

Developed using Java 21, JavaFX, Maven, and JSON file storage.



Prepared by

Course and section

Submitted to

School

Submission date

Contents

1. Introduction 3

2. Program features and technologies 4

3. Java programming concepts applied 5

4. Software Development Life Cycle 6

5. System design 7

6. Algorithms, pseudocode, and flowcharts 8

7. User interface 13

8. Testing and results 15

9. Running and demonstrating the program 16

10. Conclusion and references 17

1. Introduction

1.1 Background of the project

Managing meals, ingredients, workouts, and daily tasks often requires several separate tools. LifeSync brings these activities into one desktop application. It gives users a convenient place to organize personal wellness information and turn daily plans into manageable actions.

1.2 Statement of the problem

The project addresses three practical problems: scattered wellness records, difficulty selecting meals from available ingredients, and conflicting daily activities. A connected view of these records can help users understand their routine and plan more consistently.

1.3 General objective

To develop a Java desktop application that combines nutrition tracking, pantry and recipe management, fitness activities, and local scheduling in a clear and usable interface.

1.4 Specific objectives

Objective	Program output
Manage user information	A profile containing body measurements, activity level, and dietary preferences.
Summarize nutrition	Daily calories and protein, carbohydrate, and fat totals.
Organize food resources	Pantry stock, low-stock information, recipes, and saved favorites.
Support daily planning	Activities, meal schedules, completion status, and conflict prevention.
Provide optional assistance	AI-generated suggestions with local fallback results.

1.5 Scope and limitations

The current version runs as a local desktop application and stores records in JSON files. It includes account access, profile calculations, nutrition summaries, pantry management, recipe favorites, activities, a local calendar, and an AI chatbot. Google Calendar synchronization is outside the scope of this version.

Internet access and a configured Gemini API key are needed for remote AI responses. Local records and fallback suggestions remain available without AI. Nutrition figures are estimates. The recipe catalog and its favorite flags are shared within the installation, while primary personal records are filtered by account.

2. Program features and technologies

Module	Implemented features
Account and Profile	Registration, sign-in, password recovery using a saved code, profile editing, BMI and energy calculations.
Home	A summary of daily plans, meals, activities, pantry information, nutrition, and upcoming tasks.
Meals	Pantry-based recipes, recipe details, favorites, meal scheduling, and logging food as eaten.
Pantry	Adding, editing, deleting, categorizing, and monitoring ingredient quantities.
Activity	Workout/activity records, progress summaries, completion status, and BMI-based fitness guidance.
Plan and Calendar	Day, Week, and Month views; nested plans; editing/rescheduling; time validation; conflict detection.
AI Chatbot	Wellness-related conversation, local fallback replies, and validated plan suggestions that can be added to the calendar.
Themes	Light and dark appearances with readable labels, nutrition colors, and consistent dialog styling.

Technology used

Technology	Purpose
Java 21	Programming language and application logic.
JavaFX 21 and FXML	Desktop controls, screens, event handling, and navigation.
CSS	Colors, typography, themes, badges, and interaction states.
Gson and JSON	Reading and saving structured program data.
Maven and JUnit	Dependency management, building, and automated testing.
Java HttpClient and Gemini	Optional external AI communication.

3. Java programming concepts applied

The application demonstrates the programming concepts identified in the project requirements. The examples below refer to the current implementation.

Concept	Application in LifeSync
Classes and objects	User, Recipe, Activity, PantryItem, and NutritionEntry represent the main records.
Arrays and ArrayLists	Collections store records; arrays store fixed values and nutrient data.
Strings	Names, preferences, categories, dates, times, and AI messages are processed as text.
Methods	Reusable methods perform calculations, validation, searching, persistence, and scheduling.
Conditionals	If/switch statements classify BMI, validate input, choose themes, and detect conflicts.
Loops	Loops process collections, build calendar views, match recipes, and aggregate nutrition values.
Searching	Recipes can be searched by names, ingredients, and tags; records are located by IDs or dates.
Sorting	Services sort recipes, pantry records, and activities using appropriate comparators.
Multidimensional data	A double[n][4] nutrition matrix stores food-entry rows and four nutrient columns.
Encapsulation	Model fields are private and accessed through getters/setters and service methods.
Exception handling	Validation and error handling manage invalid input, file problems, and AI request failures.
File handling	The application reads and writes JSON records and saves the selected theme.
External API integration	The existing Gemini service provides optional AI output. Calendar scheduling remains local.

Some proposal terms use different class names in the program. For example, PantryItem represents a stored ingredient, NutritionEntry represents logged food, and Activity represents a local calendar event. A recipe's favorite status is stored on the Recipe object.

4. Software Development Life Cycle

The following stages organize the development and evaluation of LifeSync.

SDLC stage	Application to the project
1. Planning	Define the project purpose, target users, features, required technologies, and local desktop scope.
2. Requirements Analysis	Identify account, nutrition, pantry, recipe, fitness, scheduling, validation, and optional AI requirements.
3. System Design	Plan the screens, model classes, service methods, data relationships, JSON storage, and program flow.
4. Development	Implement the Java classes, JavaFX controllers, FXML views, CSS themes, calculations, and persistence.
5. Testing	Check valid/invalid input, calculations, favorites, schedule conflicts, file operations, themes, and fallback behavior.
6. Deployment	Build and run the desktop application using Java 21 and the Maven Wrapper for demonstration and evaluation.
7. Maintenance and Evaluation	Correct reported defects, improve readability and usability, and keep the program and documentation consistent.

Development outputs

The project outputs include the working desktop application, Java source code, FXML views, stylesheets, JSON record handling, automated tests, and this program documentation. Testing and evaluation support revisions to the implementation rather than introducing unrelated features.

5. System design

LifeSync separates screen presentation from data and business operations. This makes the program easier to understand, test, and maintain.

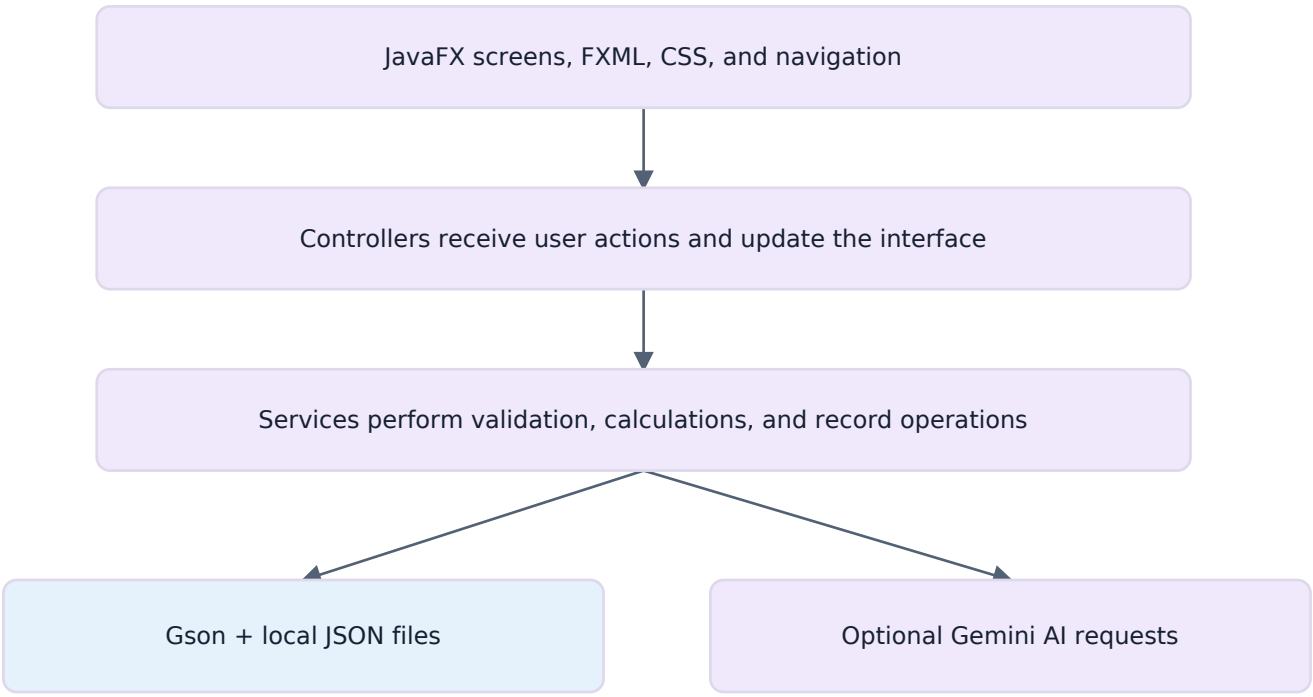


Figure 1. Main program structure and data flow.

Main data classes

Class	Main information stored
User	Identity, account details, body measurements, activity level, and dietary preferences.
Activity	Owner, title, date, start/end time, category, status, and optional parent activity.
PantryItem	Owner, ingredient name, category, quantity, unit, and low-stock threshold.
NutritionEntry	Owner, date, food/meal type, calories, protein, carbohydrates, and fat.
Recipe	Ingredients, instructions, nutrition estimates, preparation/cooking times, tags, and favorite status.

A user can have several activities, pantry items, and nutrition entries through their user ID. An activity may refer to a parent activity to represent a nested plan. Recipe records form a shared catalog. JSON files retain these records between application sessions.

6. Algorithms, pseudocode, and flowcharts

6.1 Account access

The program checks the entered username or email and compares the password hash with the saved account record. Valid credentials open the main application; invalid input displays an error.

```
LOGIN(identifier, password)
  IF a required field is empty: display a validation message
  LOAD the saved user records
  FIND the account matching the username or email
  IF the password hash matches:
    SET the current user and OPEN the dashboard
  ELSE display an authentication error
END
```

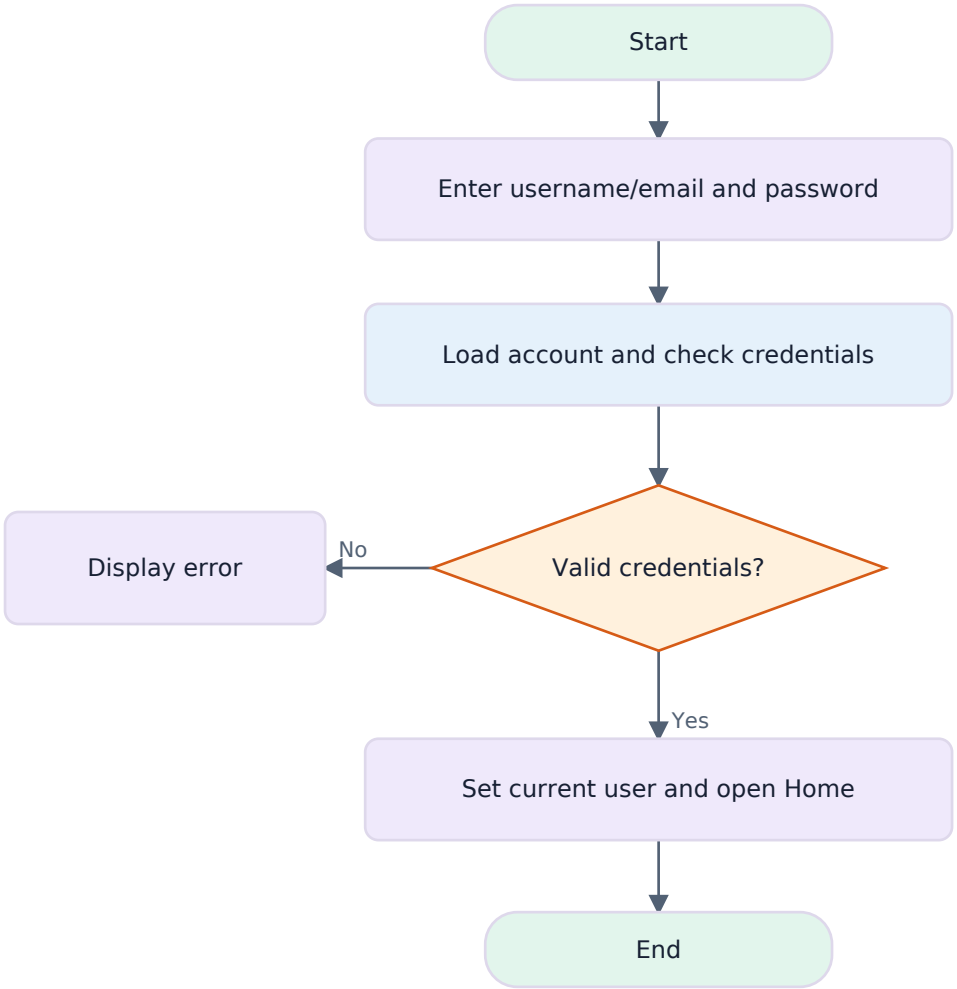


Figure 2. Login flowchart.

Registration checks required information and duplicate accounts before saving. Password recovery uses a saved single-use recovery code; successful reset consumes that code.

6.2 Profile calculations and nutrition summary

Calculation	Rule used by the program
BMI	weight in kg / (height in meters)^2; displayed to one decimal place.
BMR	10 x weightKg + 6.25 x heightCm - 5 x age, then +5 for male or -161 for female.
TDEE	BMR x activity multiplier: 1.2, 1.375, 1.55, 1.725, or 1.9.
Daily energy target	Rounded TDEE, adjusted by the program's BMI category rule: +250 for underweight; -350 for overweight/obese with a minimum of 1400; unchanged for normal weight.

The implemented BMI categories use boundaries of 18.5, 25, and 30. Missing body measurements produce a pending display. These values support the application's estimates and suggestions.

Two-dimensional nutrition data

Each row of the nutrition matrix represents a food entry for the current user and date. The columns store calories, protein, carbohydrates, and fat. Column totals feed the Home dashboard.

Food entry	kcal	Protein (g)	Carbs (g)	Fat (g)
Entry 1	420	35	40	12
Entry 2	180	10	20	8
Total	600	45	60	20

```
SUMMARIZE_NUTRITION(user, date)
  FILTER food records by user and date
  STORE each entry in matrix[row][calories, protein, carbs, fat]
  SET totals = [0, 0, 0, 0]
  FOR each row in the matrix:
    FOR each nutrient column:
      ADD the value to its column total
  CALCULATE macro calorie shares using 4, 4, and 9 kcal per gram
  DISPLAY totals and colored nutrition bars
END
```

The macro bars show the share of macro-derived calories. They are not separate macro goals. The program handles an empty day without dividing by zero.

6.3 Pantry, recipes, and favorites

Pantry quantities must be valid non-negative numbers. Recipe services search names, ingredients, and tags, compare ingredients with pantry records, and support sorting by criteria such as name, calories, preparation time, and popularity. The Meals screen displays pantry-based recipes and a searchable favorites/pantry catalog.

```
SELECT_AND_SAVE_RECIPE(user)
  READ the user's available pantry ingredients
  GENERATE local recipe suggestions; optionally request AI suggestions
  DISPLAY recipe cards and details
  WHEN Favorite is selected:
    INSERT the displayed recipe into the catalog if it is new
    TOGGLE its favorite status and SAVE the catalog
  WHEN Add to Plan is selected: validate the meal schedule before saving
END
```

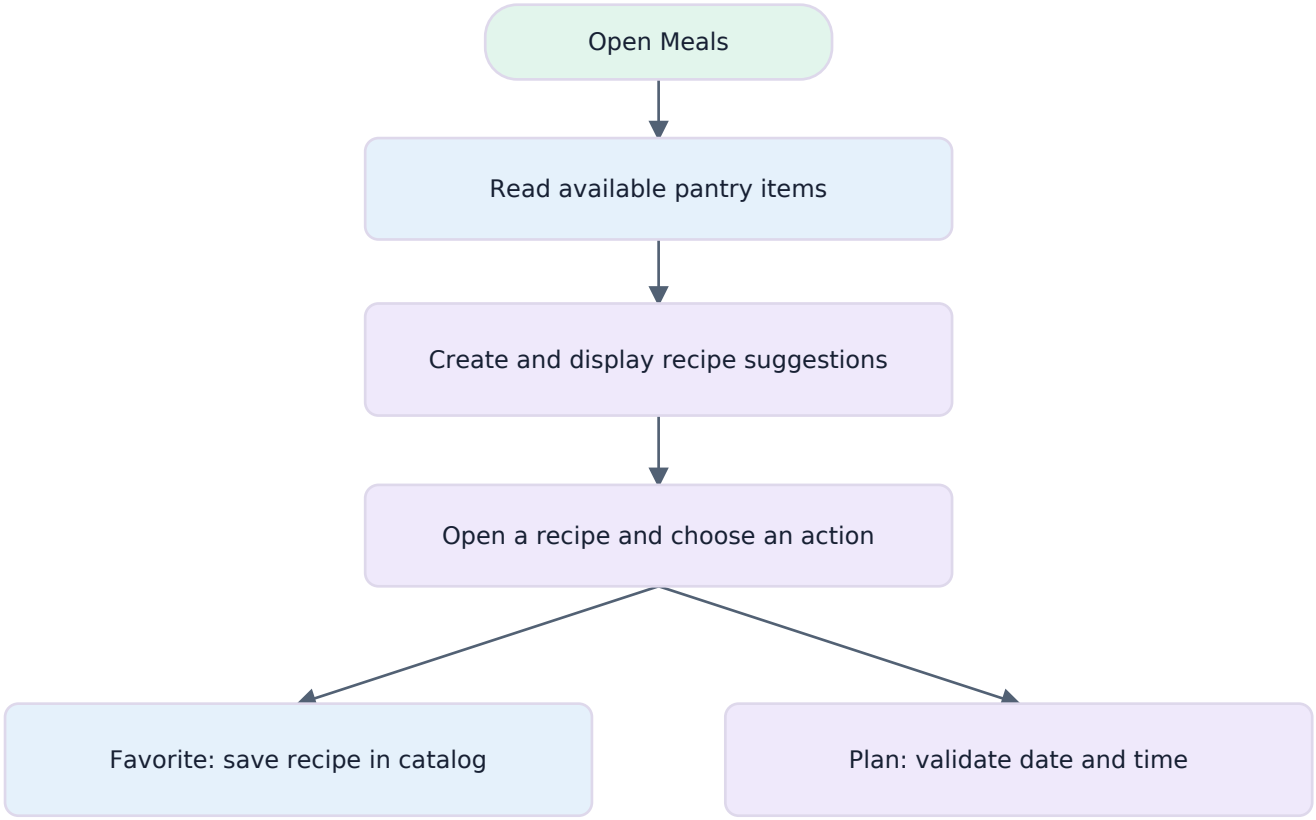


Figure 3. Recipe selection and the two main actions.

Pantry matching compares ingredient names. It does not guarantee sufficient ingredient quantities or certify dietary/allergen suitability.

6.4 Scheduling and conflict detection

Activity times are converted to minutes after midnight. Starts range from 00:00 to 23:59; 24:00 is accepted only as an end time. A proposed interval conflicts when it starts before an existing interval ends and the existing interval starts before it ends.

```
SAVE_ACTIVITY(proposed)
  VALIDATE the title, date, start time, and end time
  CHECK parent bounds when the activity is a sub-plan
  READ activities for the same user, date, and scheduling level
  FOR each relevant existing activity:
    IF proposed.start < existing.end AND existing.start < proposed.end:
      REJECT the overlapping activity
  IF no conflict exists: SAVE and refresh the calendar
END
```

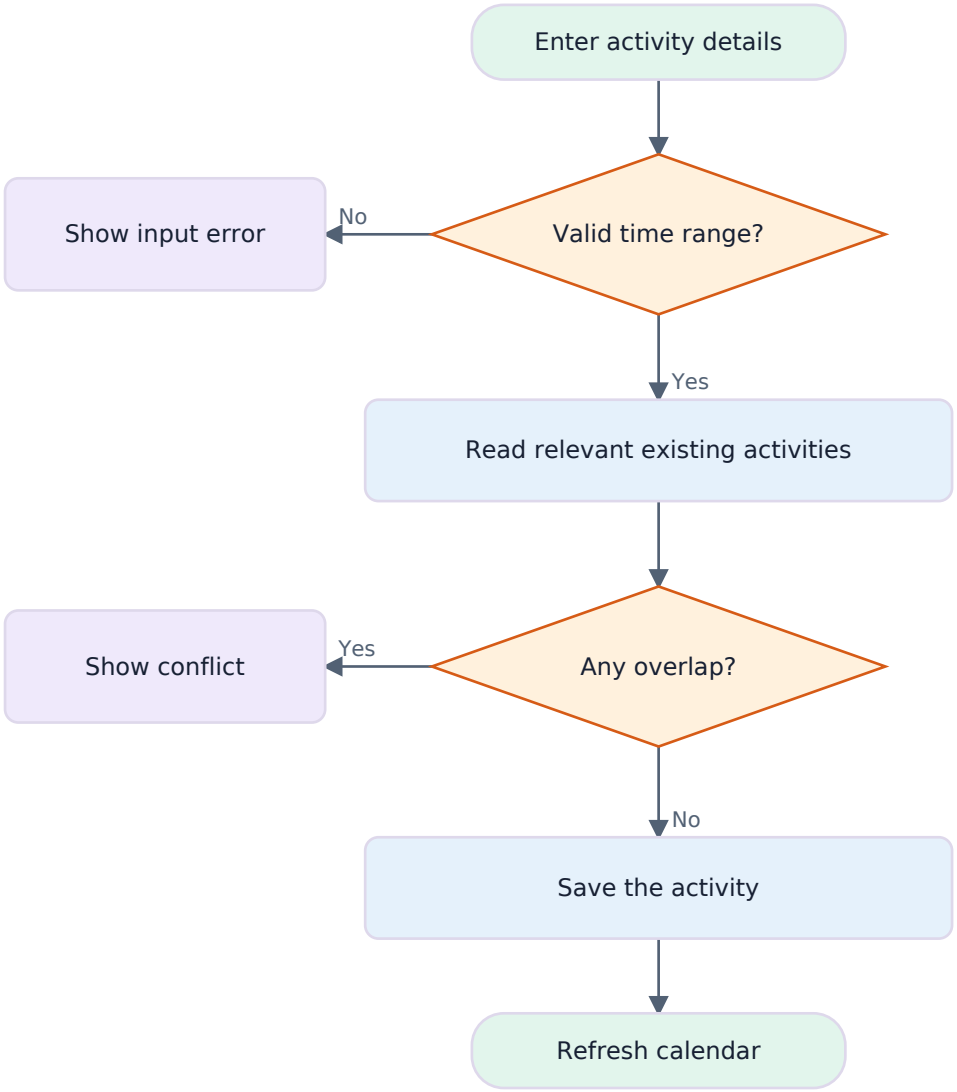


Figure 4. Validated scheduling flowchart.

Adjacent activities are allowed. For example, 22:00-23:00 and 23:00-24:00 do not conflict. Sub-plans may overlap their parent but are checked against siblings. Generated multi-event plans are checked completely before saving.

6.5 AI assistance and file handling

Gemini is an optional external service. A local result is prepared first, and network requests run asynchronously so the interface stays responsive. Invalid or unavailable AI responses use the local fallback.

```
REQUEST_SUGGESTION(context)
  PREPARE a local fallback
  SEND an asynchronous AI request
  IF a usable response is received: use the response
  ELSE use the fallback
  UPDATE the interface on the JavaFX thread
END
```

Chat behavior

The chatbot accepts LifeSync-related questions and maintains the conversation while switching tabs. Switching accounts prevents the previous user's history from appearing in the new account. Recognized plan suggestions can be added to the local calendar after all items pass validation.

JSON persistence

User records, activities, pantry items, nutrition entries, recipes, and theme settings are stored as JSON. DataStore handles the main record files, while services decide which records to add, retrieve, update, or remove.

```
SAVE_RECORDS(filename, records)
  CREATE the data folder if necessary
  WRITE the records as UTF-8 JSON to a temporary file
  REPLACE the destination after the write completes
  REPORT an error if reading or saving fails
  REMOVE any remaining temporary file
END
```

Situation	Program response
Invalid user input	Display validation feedback and prevent an invalid save.
Overlapping plan items	Reject the conflicting activity or the entire generated batch.
AI request fails	Use the prepared local suggestion.
File operation fails	Report the error rather than claim that the save succeeded.

7. User interface

The existing screen arrangement is used in both themes. Summary cards and colored macro indicators help users recognize the most important daily information.

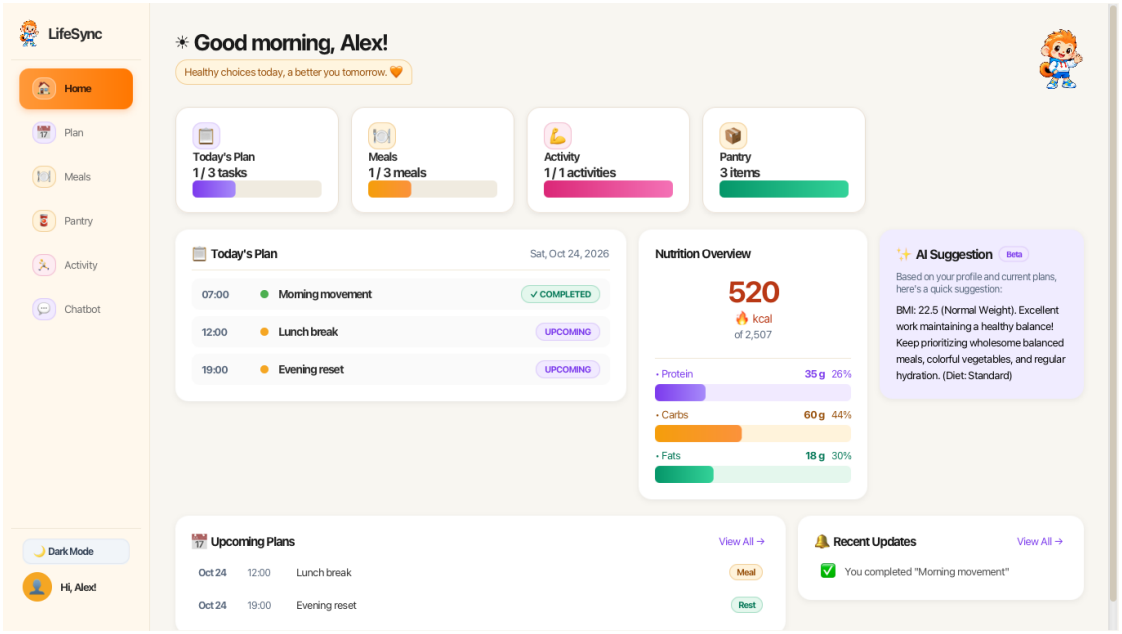


Figure 5. Home dashboard in light mode.

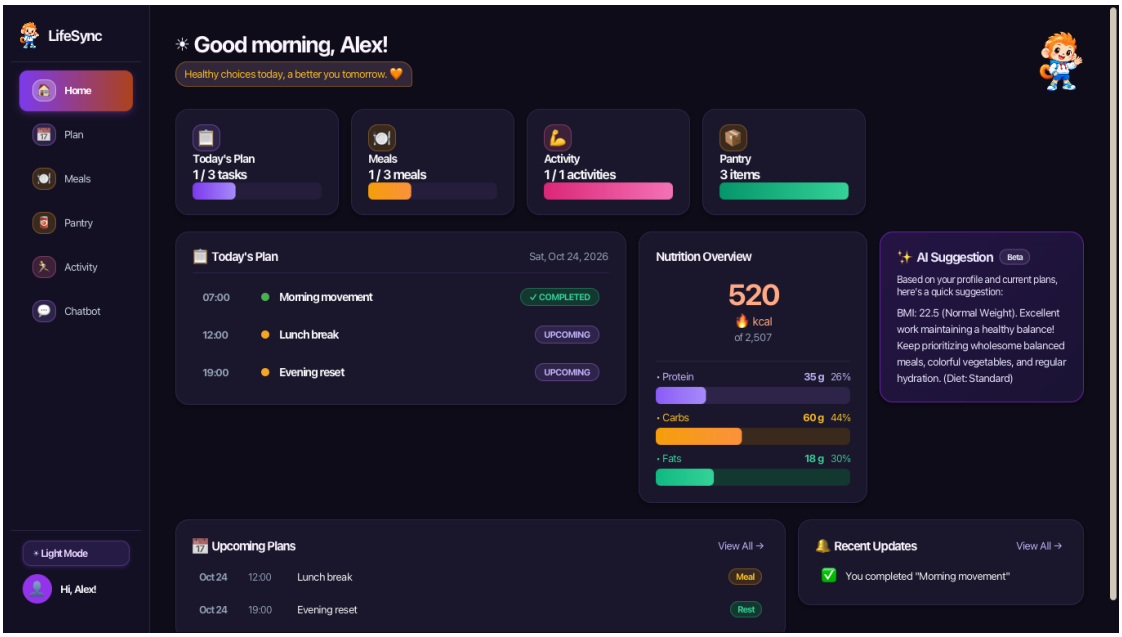


Figure 6. Home dashboard in dark mode.

The sidebar opens Plan, Meals, Pantry, Activity, and Chatbot. The profile area opens My Profile. The theme button changes the appearance without changing the workflow.

7.1 Meals and recipe details

Recipe cards provide a quick view of suggested meals. Opening a card displays ingredients, cooking instructions, nutrition estimates, and actions for favorites and scheduling.

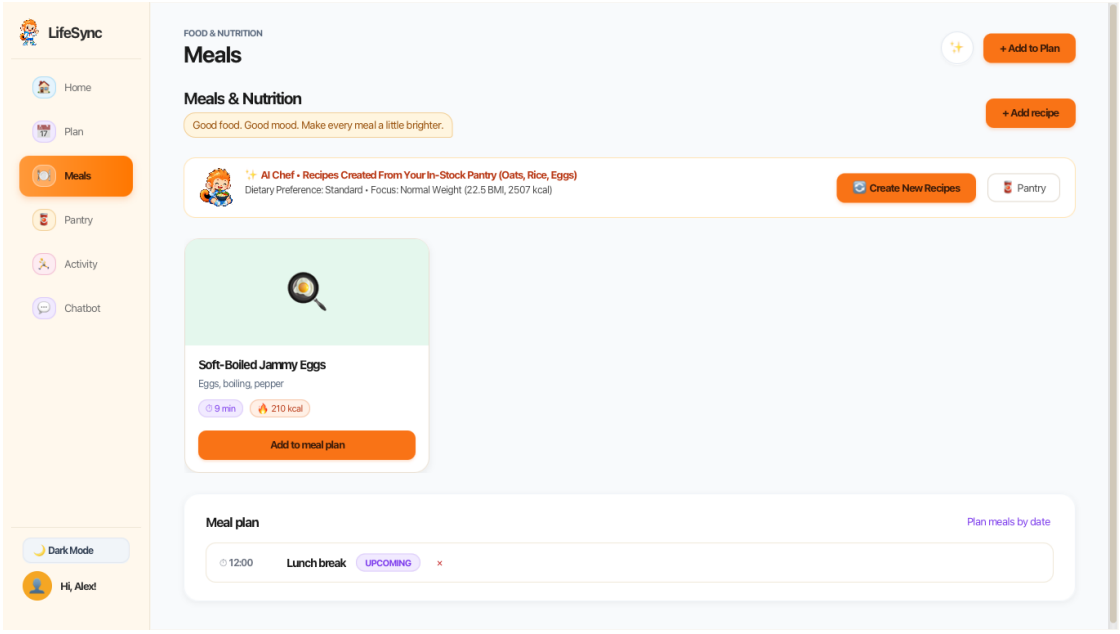


Figure 7. Pantry-based recipe suggestions and the meal plan section.

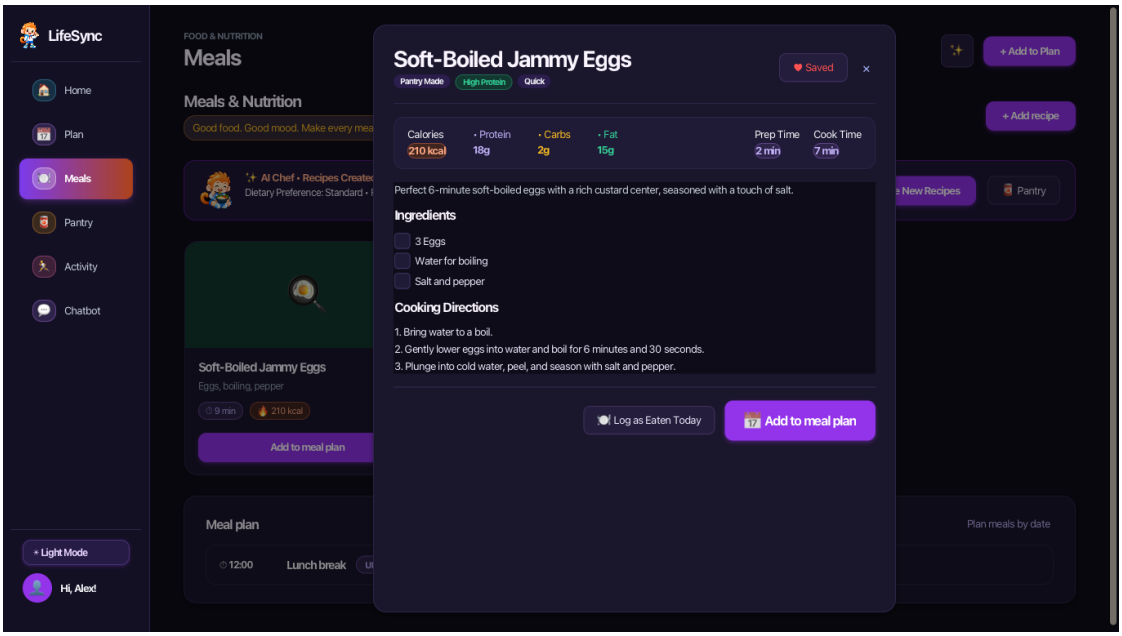


Figure 8. Recipe details, macro values, and favorite action.

Screenshots use demonstration records. The displayed information changes with the signed-in user's profile, pantry items, food logs, and activities.

8. Testing and results

The latest recorded Maven test run completed **75 tests with zero failures and zero errors**. The run recorded 0 skipped tests. Tests check the program logic and persistence; desktop checks evaluate the visible screens and interactions in both themes.

Test area	Tests	Result
Time intervals and midnight boundaries	5	Passed
FXML structure	1	Passed
Transparent mascot assets	1	Passed
Scheduling, hierarchy, and plan batches	21	Passed
Authentication and password recovery	9	Passed
Chat topic validation	4	Passed
JSON file operations	3	Passed
AI results and fallback behavior	7	Passed
Pantry quantity validation	5	Passed
Recipe searching, sorting, and favorites	8	Passed
Nutrition matrix and generated favorites	7	Passed
Theme state and persistence	4	Passed

Representative validation cases

Input or action	Expected outcome
A plan from 23:00 to 24:00	Accepted when it does not overlap a peer activity.
An overlapping plan	Rejected with a conflict message.
A negative or non-finite pantry quantity	Rejected without changing the saved stock.
Favorite a generated recipe, then reload	The favorite is retained in the catalog.
View long fitness advice in dark mode	Text remains readable and scrolls with the page.

9. Running and demonstrating the program

9.1 Requirements and startup

Use Java 21 and open the project folder. Maven downloads the required dependencies during initial setup. The optional AI features require a Gemini API key in the run environment.

```
Windows PowerShell:
  .\mvnw.cmd javafx:run

macOS (included launcher):
  ./run.sh

Run automated tests with Java 21 selected:
  .\mvnw.cmd test     (Windows)
  sh mvnw test        (macOS/Linux)
```

9.2 Suggested classroom demonstration

Step	What to show
1. Account and Home	Sign in with a demonstration account and explain the dashboard summaries.
2. Profile	Show body measurements, BMI classification, preferences, and calculated energy values.
3. Pantry and Meals	Add an ingredient, view a recipe, save a favorite, and log a meal.
4. Plan and Activity	Create an activity, show the calendar views, and demonstrate a rejected overlapping plan.
5. Themes	Switch between light and dark mode and show that the same controls remain available.
6. Optional AI	Ask a relevant question or show a suggested plan. Explain that a local fallback is used if the service is unavailable.

Save the recovery code provided during registration. Recorded data and theme settings remain after restarting the program; users sign in again, and chat history starts a new application session.

10. Conclusion and references

LifeSync combines several everyday wellness and planning tasks in a single Java desktop application. Its implementation demonstrates object-oriented modeling, arrays and collections, strings, reusable methods, conditionals, loops, searching, sorting, multidimensional data, encapsulation, exception handling, file persistence, and optional API communication.

The current program supports personal profiles, nutrition summaries, pantry and recipe management, favorites, activity tracking, and conflict-aware local scheduling. The interface provides light and dark themes, while automated and desktop testing support the reliability of these workflows.

The project remains a local desktop application. AI access depends on the configured service, nutrition figures are estimates, and Google Calendar synchronization is not included. These boundaries define the version presented in this submission.

Project references

Material	Use in this documentation
LifeSync Java source code	Program features, calculations, classes, algorithms, and validation behavior.
FXML views and CSS stylesheets	Screen arrangement, navigation, appearance, and theme behavior.
pom.xml and Maven Wrapper configuration	Java version, frameworks, dependencies, and build commands.
JUnit test reports	The recorded 75-test result and test coverage summary.
Project requirements provided for LifeSync	The Java programming concept list and SDLC stages.